PROJECT PROPOSAL · FOR FACULTY APPROVAL

LabTrack

Hardware Component Availability, Issuance & Tracking System

Turning fragmented laboratory hardware handling into a traceable digital lifecycle.

SY · Semester 3 · AIML 2025–29

CCE Presentation & Viva

DCDS Lab ESE Miniproject

BEFORE WE BEGIN

What we are asking you to approve

This is a proposal, not a finished system. Every statement is labelled so that nothing we assumed is mistaken for college policy.

● Observed / confirmed

Things the project team has directly observed in the current lab process, or that come from an official rubric.

● V1 assumption

Temporary working rules so development can start. They are configurable and will be replaced by your answers.

● Pending faculty confirmation

Institutional decisions only faculty / college can make. Collected as questions at the end of the deck.

● Proposed — for approval

Our technical design choices: architecture, data model, tech stack. Open to feedback.

● Future scope

Ideas we discussed but are deliberately **not** building in V1.

Core principle

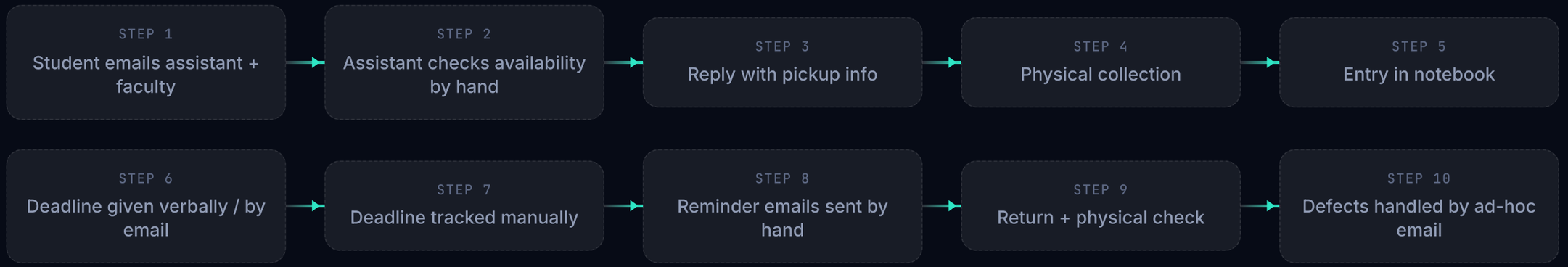
Automate the work lab assistants already do by hand, **without changing departmental policy** unless that policy is explicitly authorised.

THE PROBLEM

Observed by team

Today, lab hardware runs on email, notebooks and memory

The process we observed for a student project that needs components:



Inventory

Partly maintained in **Excel**, updated manually.

Issue history

Partly kept in **physical notebooks**.

Reservations

No unified queue for components that are already issued.

CONSEQUENCES

Everyone pays for the missing system

Observed by team

Lab assistants

- Manual availability checks
- Repeated email threads
- Notebook entries
- Tracking many due dates
- Manual reminders
- Defect / loss records by hand

Students

- Can't see what's available
- Don't know which lab has it
- Don't know whom to ask
- No request status
- No reservation queue
- No borrowing history

Faculty

- Requests buried in email
- Project info fragmented
- Approval status not central
- Overdue cases need manual follow-up
- No defect / penalty view

Department

- Records split across Excel and notebooks
- Weak traceability
- Unmet demand is invisible
- No utilisation insight
- Weaker accountability

THE PROPOSAL

One system for the entire component lifecycle

LabTrack is **not** an inventory CRUD app. It digitises the full journey from project requirement to return and accountability.



No performance-improvement numbers are claimed. Impact will only be reported if measured.

SCALE

Sized for one department, built to move

Approximate, per team

8–10
Labs

10–12
Lab assistants

700–800
Students

5–10
Expected concurrent users

Prototype first

V1 is a working prototype for demonstration and evaluation. Department adoption is a goal, **not** an approval we have.

Web first

A responsive web application. A native mobile app and Play Store release are future scope.

Portable

Demonstrated on a cloud VM with synthetic data, and designed to move to college infrastructure if that's approved.

WHO USES IT

Four roles, each with clear authority

Student

- Create project, add team
- Pick faculty mentor
- Search catalogue & availability
- Request / reserve
- Track due dates, extend
- History, penalties, REPO

Faculty

- Sees **only** relevant projects
- Approve / reject / modify
- Overdue notifications
- Escalation & extensions
- Defect / penalty visibility

Lab Assistant

- **Authority over physical inventory**
- Verify & issue items
- Pick the actual asset
- Set duration
- Inspect returns, log defects
- Transfers, FIFO override

Admin

- Users, roles, labs
- Components & assets
- Config & Excel import
- Logs & demo data
- Role can be **handed over**, not tied to one person

HoD role in the approval chain: UAQ5

The HoD's role in approvals and escalation is still to be confirmed. See the questions section.

THE CORE IDEA

One project, one traceable lifecycle

Requests are tied to a **project** (team + mentor + requirements), not sent as one-off student emails.

01
Project created
team + mentor

02
Requirements
from catalogue

03
Availability check
qty · lab · assistant

04
Request
available items only

05
Approval
faculty workflow

06
Physical issue
by lab assistant

07
Active tracking
due date set

08
Reminders
scheduled

09
Extension
separate record

10
Reservation
FIFO queue

11
Return
student hands back

12
Inspection
condition + evidence

13
Defect / penalty
if applicable

14
History
never overwritten

■ Request & issue ■ Accountability while issued ■ Return & closure

DATA MODEL

Component Type ≠ Physical Asset

Not every resistor needs a serial number, but every Arduino needs its own history.

COMPONENT TYPE

The general item

Arduino Uno · ESP32 · Ultrasonic sensor · Resistor · LED · Jumper wire

Holds category, description, technical specs and alternatives.

PHYSICAL ASSET

One real unit

`ARDUINO-001` · `ARDUINO-002` · `ESP32-014`

Has its own current lab, status, issue history, defects and transfers. Internal ID is unique; the manufacturer serial is optional.



Individually tracked

Arduino, ESP32, Raspberry Pi and other higher-value boards where unit-level history matters.



Quantity tracked

Resistors, capacitors, LEDs, jumper wires: only counts are tracked.



Consumable

Items not expected back. Exact classification: UAQ7.

REQUEST LOGIC

Unavailable items are never sent as normal requests

V1 rule

Available

Shows quantity, current lab, responsible assistant and specs.

→ Student can request it

Exists but currently issued

Shows status and expected return.

→ Join the FIFO reservation queue

The assistant can override the order, but must record who, why and when.

Not stocked

Shows unavailable and suggests alternatives.

→ If there's no usable alternative, log it as unmet demand

This is demand insight only, not procurement.

AVAILABILITY STATES

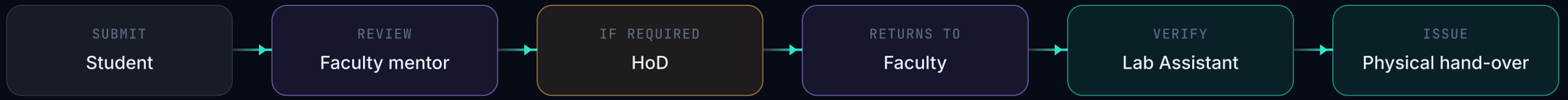
Available | Insufficient qty | Currently issued | Reserved | Defective | Under repair | Out / unavailable | Not stocked

WORKFLOW

Workflow: UAQ5

Approval is configurable. Physical issue stays human.

The temporary V1 route, which will be replaced by whatever policy you confirm:



Faculty can

- Approve
- Reject (student is notified)
- Modify quantities / items

At issue, the assistant

- Physically verifies the items
- Selects the actual asset (e.g. ARDUINO-007)
- Sets the duration, which creates an **issue transaction**

Open questions

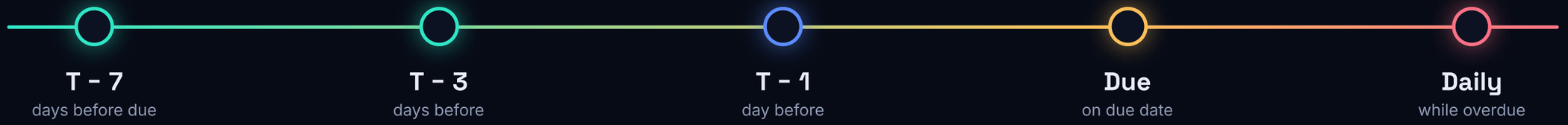
- Is HoD approval always needed?
- Do multiple mentors have equal authority?
- Can approval be revoked?

The route is stored as configuration, so changing policy won't need code changes.

ACCOUNTABILITY

Reminders run automatically

Timings: UAQ12 / UAQ16



Channels

In-app notifications, plus automated email from a dedicated project address. Reminders stop on return or on an approved extension.

Extensions

A separate record: request → approval → assistant updates the due date. History is kept, and approved time doesn't count as late.

Overdue escalation

The assistant monitors and decides whether to notify faculty. Faculty decide whether to escalate to the HoD. This is human judgement, not automatic.

CLOSURE

Every return is inspected and recorded

Student returns → Assistant inspects → Condition recorded → Photo evidence if needed → Inventory updated → Student acknowledged

Defect / damage / loss

Records the asset, student, project, original issue, who found it, description, evidence and repair status. The asset moves to Defective, Under repair or Unavailable. **History is kept.**

Penalties

Records for late return, damage or loss. The assistant records them; faculty or admin can waive or reduce. No payment gateway.

- Amounts / formula: UAQ13

REPO: reputation score

Proposed. Built from on-time returns, late returns, damage and extensions. It must be explainable, transparent and auditable.

- Should it exist? UAQ15

INTELLIGENCE

AI helps students choose. The database decides.

AI is deliberately a small part of the system. It is not what the project is about.

AI may

- Explain what a component is and does
- Interpret a student's project requirement
- Suggest possible alternatives
- Help students compare component choices

Suggestions are grounded in the real catalogue, so only items that actually exist are shown.

AI may never

- Decide whether an item exists
- Override inventory or status
- Issue components or approve requests
- Change reservation order
- Decide penalties
- Act as institutional authority

Unmet-demand insight: repeated requests for unstocked items are grouped together so the department can see what's in demand. Plain database aggregation, no AI needed.

ARCHITECTURE

Python backend, SQL + NoSQL, web client

Proposed — for approval

CLIENTS

Web UI
React SPA, with role-specific dashboards for Student, Faculty, Lab Assistant and Admin

Email inbox
Automated mail from a dedicated project address

EXTERNAL

SMTP service
Outbound reminders & notifications

LLM API
Advisory only, behind an adapter

Excel files
Import of existing inventory

Python API · FastAPI

REST · JSON · JWT

Auth & RBAC
Replaceable auth, enforced on the server

Projects & Teams
Members, mentors, requirements

Catalogue & Assets
Types, units, tracking modes

Workflow engine
Configurable approval routing

Issue & Return
Transactions, inspection, defects

Reservations
FIFO queue + audited override

Scheduler
Reminder & overdue jobs

Notifications
In-app + email dispatch

Analytics
Demand, utilisation, history

AI adapter
Grounded in catalogue, read-only

DATA

PostgreSQL
System of record for transactional data: users, projects, components, assets, issues, reservations, extensions, penalties.

Constraints · joins · stored procedures

MongoDB
Event and document data: audit trail, notifications, unmet-demand events, AI interaction logs, inspection evidence metadata.

Aggregation pipelines

Deployment
Cloud VM with synthetic data → college VM if approved

DATA STRATEGY

Proposed — for approval

Each database does the job it is best at

Not bolted on for marks: the split follows the shape of the data.

PostgreSQL: must be consistent

- Two students can't be issued the same asset (unique IDs, constraints, row locks)
- **Stored procedure** for issuing: check availability, lock the asset, create the issue, all in one transaction
- **Joins**: project → members → requests → issues → assets → lab
- **Subqueries / GROUP BY**: overdue issues per lab, students with repeated late returns

MongoDB: append-heavy, flexible

- Audit events with differing payloads (approval, override, transfer, role change)
- Notification history per user
- **Aggregation pipeline**: most-requested unstocked components per month
- **Aggregation pipeline**: reservation wait time and return delay distributions

Rule: PostgreSQL is the single authority for inventory and workflow state. MongoDB never decides availability; it records and analyses what happened.

SECURITY

Least privilege, full audit trail, no real PII off-campus

Access control

- Server-side RBAC on every endpoint
- Students see only their own projects
- Faculty see only their mentored projects
- Assistants see only their assigned labs

Input & data safety

- Validation on every input
- Parameterised queries only (no SQL injection)
- Hashed passwords, rate-limited login
- Restricted evidence uploads
- APIs return only the fields needed

Auditability

- Approvals, issues, overrides, transfers, penalties and role changes are all logged
- Transactions are never silently overwritten
- Admin is a transferable role

Cloud demo uses synthetic data only

No real student PII on external servers until college privacy policy is known (UAQ17).

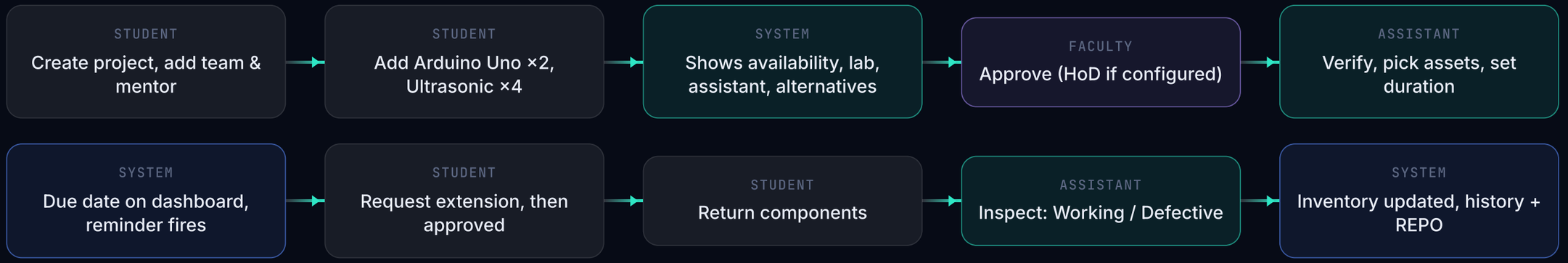
Authentication is replaceable

V1 uses local login: PRN for students, institutional email for staff. College SSO can be added later (UAQ1).

LIVE DEMONSTRATION

One project, start to finish

What we will show live on the cloud deployment:



Exception: already issued

A second student sees "Currently issued" and reserves the item. On return, the first person in the FIFO queue is offered it.

Exception: not stocked

The system suggests an alternative from the catalogue, or logs unmet demand that appears on the analytics dashboard.

ACADEMIC RELEVANCE

One application covers both CCE and DCDS

DCDS LAB ESE · 10 MARKS · 40%

CRITERION	MARKS	IN LABTRACK
Application design	2	Full role-based lifecycle, modular services
Python implementation	2	Classes, exceptions, modules, Excel file import
SQL + NoSQL integration	2	PostgreSQL + MongoDB with a clear split
Advanced querying	2	Joins, subqueries, stored procedures, aggregation pipelines
Code & documentation	1	Docs, clear UI, structured repo
Testing & security	1	Unit/API tests, validation, RBAC

CCE · 30 MARKS · CO3 / CO4

COMPONENT	MARKS
Topic submission	5
Research / literature survey	5
Presentation skills	5
Running example (live cloud)	10
Viva	5

TIMELINE

Working back from the DCDS ESE (26–30 Oct 2026)

Proposed plan

WEEK 1

Foundation

- Faculty answers to P0 questions
- Requirements & schema docs
- DB schema, auth, RBAC
- Catalogue, assets, Excel import

WEEK 2

Core lifecycle

- Projects & teams
- Request → approval → issue
- Issue stored procedure
- Role dashboards

WEEK 3

Accountability

- Reminders, extensions
- FIFO reservations
- Return, inspection, defects
- Audit + Mongo analytics

WEEK 4

Harden & ship

- AI advisory layer
- Tests, validation, security
- Cloud deployment
- Docs + demo rehearsal

If time runs short, these are cut first: REPO scoring, AI suggestions and asset transfers. The core lifecycle and the database requirements are never cut.

SCOPE DISCIPLINE

What we are deliberately not building

Future scope

Native mobile app / Play Store	Cabinet / shelf / drawer locations	Full procurement workflow	Payment handling
Formal dispute resolution	Enterprise accounting	Full college SSO (until confirmed)	Cross-department sharing
Warranty management	Advanced analytics / AI	Full notebook-history migration	Real student data on cloud

V1 is deliberately small and complete rather than big and half-built. Everything here can be added later without changing the core data model.

DECISIONS WE NEED FROM YOU

Policy questions that change the data model

● P0: needed before build

UAQ5 · APPROVAL

Exact approval route

Is HoD approval always required? Equal-authority mentors? Can approval be revoked? Who is the final approver before the assistant?

UAQ12 · BORROWING

Duration & extensions

Official borrowing period, extension rules, maximum extensions, who approves them.

UAQ13 · PENALTIES

Late / damage / loss

Rules, formulas, replacement cost, who can waive or reduce, appeals.

UAQ16 · REMINDERS

Reminders & escalation

Official timings, escalation thresholds and recipients, HoD involvement.

UAQ7 · TRACKING

Item classification

Which items are individual, quantity-tracked or consumable (wires, resistors, LEDs…)?

UAQ8 · IDs

Existing identifiers

Which assets have serials, department IDs or asset codes that we must keep?

UAQ1 · AUTH

Authentication / SSO

Which college identity system exists, and can LabTrack integrate with it?

UAQ2 · MASTER DATA

Student / staff datasets

Can we get authorised read access, or should we use synthetic data?

UAQ17 · PRIVACY

PII & external cloud

Policies for student data, cloud hosting, backups, email and logs.

UAQ18 · INFRA

College infrastructure

Is a server/VM, database, DNS, HTTPS or internal network available for production?

ACADEMIC · TEAM STRUCTURE

DCDS team-size requirement

The DCDS rubric expects a 4-member team from the same division. Two members are currently building the software. How should this be handled for registration?

OPERATIONAL & GOVERNANCE

P1 / P2

Questions we can work around for now, but need answered

UAQ3 · P1

Staff IDs

Do faculty and assistants have institutional IDs besides email?

UAQ4 · P1

Lab ↔ assistant mapping

Multiple assistants per lab? Multiple labs per assistant?

UAQ6 · P1

Unmet demand

What to record, who reviews it, should it feed procurement discussion?

UAQ10 · P1

Asset transfers

Who authorises moving equipment between labs?

UAQ11 · P1

Reservations

How long a reservation stays valid; what happens if it isn't collected; category-level reservations?

UAQ14 · P1

Return inspection

Which parameters must the assistant record?

UAQ15 · P1

REPO score

Should it exist? Formula, effects, override, challenge process?

UAQ19 · P1

Retention

How long to keep backups, audit logs, issue and penalty history?

UAQ20 · P1

Cross-department

Can equipment be shared between departments? Who owns it?

UAQ21 · P1

Demand recipients

Who should see unmet-demand reports?

UAQ22 · P2

Long-term ownership

After adoption, who maintains, administers and recovers the system?

UNTIL ANSWERED

Our default

We use the V1 assumption, make it configurable, and label it as pending.

APPROVAL REQUEST

What we need today

1 · Approve the topic

LabTrack as our unified CCE and DCDS project.

2 · Approve the architecture

Python (FastAPI) + PostgreSQL + MongoDB + web UI, deployed on a cloud VM with synthetic data.

3 · Answer or route the PO questions

Or point us to the person who owns each policy (lab in-charge, HoD, IT).

4 · Allow lab-assistant input

A short session with one lab assistant to validate the workflow and inventory classification.

LabTrack turns fragmented laboratory hardware handling into a traceable digital lifecycle, from requirement to return.

LABTRACK · PROJECT PROPOSAL

Thank you.

Questions & feedback welcome.

- Observed
- V1 assumption
- Pending faculty confirmation
- Proposed
- Future scope